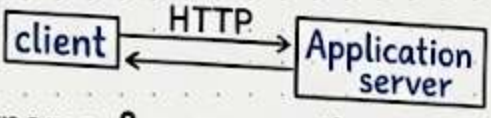
Lecture 10 : Authentication & Authorization system.

Authentication : who you are ? what is your identity.

Authorization : what you can do ?

Access Control / access policies



Authenticate : Login (username & password)

Flow :

user comes to our platform
user go to register himself / herself (signin)
user go to Login (Authentication)
Now you can access application server, based on your policies / rules (Authorization)

Requirements :

1. User Registration : User can register on our Platform (basic details → name , no, etc)
2. User Login : Username & password (Authentication)
3. 1 FA va MFA (Multi factor Authentication) :
   Secure applications → banking / Gmail etc.
4. Password Recovery : (Forgot password)
   Notification → mobile / email (Already registered)
5. Session Management : stateless vs stateful
   stateful architecture
6. Access Control (Authorization)
   Leetcode → Premium feature
   user level → key : IsPremiumUser
   API level → key

Questions :

Q: MFA / 1FA ?
↳ MFA (Notification → email link)

Q: Application → Daily Active user ?
↳ 100k DAU

Q: Session management ? Stateful vs stateless ?
↳ stateless

→ Back of the envelope calculation

user traffic : 100k / day (DAU)

QPS : 100,000 / 86,400 = 1.15 req/sec (signup/signin)

Storage unit : Register → (username, email, mobile)

1 user → 5 KB (user info)

estimate → 5 years

⇒ $100000 \times 365 \times 5 \times 5 \times 2^{10}$

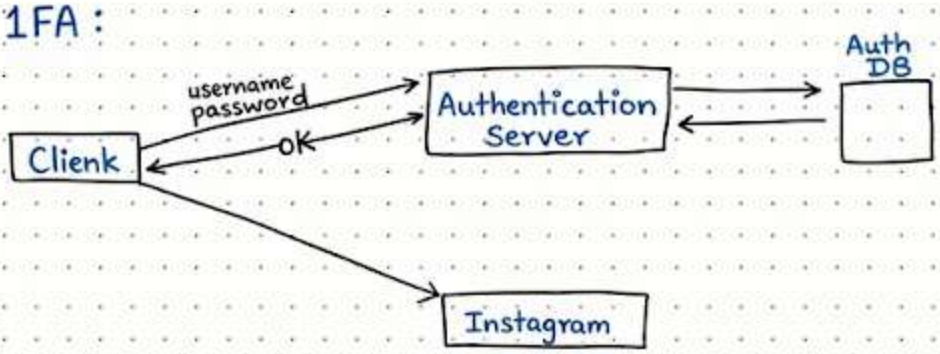
⇒ 912500000×2^{10}

⇒ 912.5×2^{30}

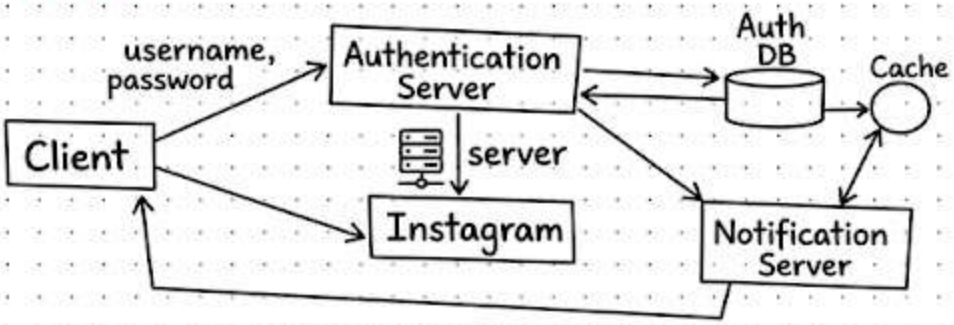
⇒ 913 GB

1FA vs 2MFA :

1FA :



MFA:

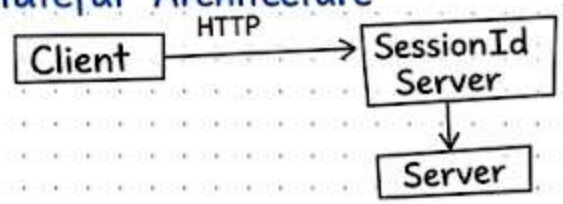


Session Management stateless

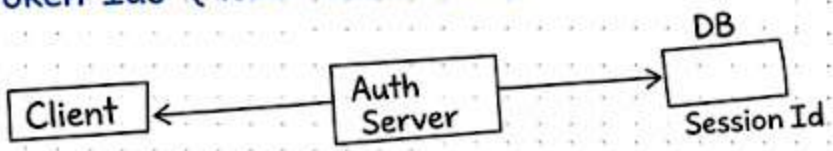


HTTP: Stateless (No info about the sender)
↳ Always new request

Stateful Architecture



Token Ids (session based Auth)
vs
Token Ids (Token based Auth)



browser cookies

DB: Table → Session Table
user SessionId
TTL (15 min)

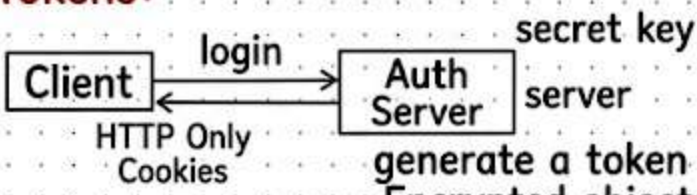
HTTP/HTTPS
Headers . SessionId

Session → Connection

Token based Authentication → JWT

More secured way

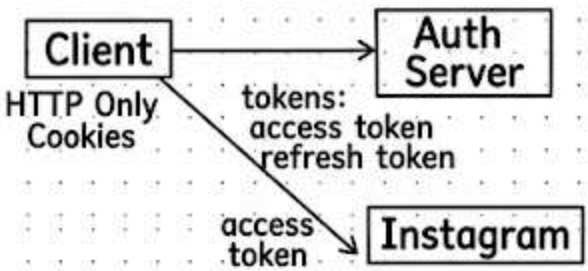
Tokens:



generate a token
Encrypted object:
- user info (name, email)
- permissions
- ttl (time to live)

Types of tokens:

1. Access token
2. Refresh token



1. Every request after authentication will come with access token.
2. Now our application server will decrypt the access token. (using its own secret key)
3. checks the TTL of access token
4. If not expired, application server will check user permission.
5. then user is allowed to interact

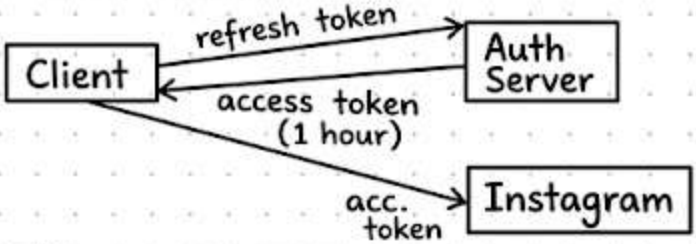
Refresh token:

Access token → TTL (Time to live)

Time → short time (1 hour)

Refresh token → TTL (Long → 1 day)

If Access token is expired, user will generate a new token.



Tokens : JWT Tokens
Get /user / profile
Authorization : Bearer ey123adB121adB

Q. Why do we even even need 2 tokens ?

Access token → TTL (Long)

Access token → will be sent with each HTTP request.
Hacker → access token (it will have limited time to exploit)

Refresh token → never sent to API calls. Only when new token needs to be generated.

Gmail : Web Browser

login in Gmail → access token valid for 1 hour
you leave your browser unattended (3 hours)
Gmail ← Browser (auto, refresh)
Comeback → Gmail silently uses refresh token to get a new access token.

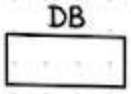
after few days : refresh token is expired
Gmail → sign in again

How passwords are stored in your Application DB ?

login

1st way - store plain text

```
{
  Username : "aditya"
  Password : "aditya123"
}
```



Storing passwords Hashes (2nd - store hashed passwords)

Password → hash (Password) → qwer1234 → save to DB

What is Rainbow table ??

A huge data set of commonly used Passwords

3rd way to store passwords :

Salt & pepper technique

password → aditya 123

hash (aditya) = qwer 1234

Salt : random string (tyu78) = qwer1234tyu78

Store the salted password in DB

hashpass	salt
qwer1234tyu78	tyu78

Pepper ; what is pepper?

pre defined string : asdf123 (at a very secured location)

aditya123 → aditya123tyu78 → asdf123aditya123tyu78

hashpassword
asdf123aditya123tyu78

plain password → plain password + salt → hash
+ pepper

↳ aditya 123 + (salt ?) = hash

Store hash to DB

Good hash functions?

password → hash (password)

hash function → slow not fast

basic hash function → SHA-1 , SHA-256, MD5 etc.
they can hash billions of record in a second

Rainbow attack

password → hash (password) → (micro seconds)

Why slow hash functions are good?

Slow hash functions make it infeasible for a attacker to have rainbow attack.

Good hashing Algo → Argon2, bcrypt, scrypt, etc.

slower (mili seconds)

user experience will not be impacted but hacker experience will be impacted

brute force attack → infeasible

Authorization : What can you do? Access Rights

1. Role based access control (RBAC)
2. policy based access control (PBAC)
3. Access control List (ACL)

RBAC → Roles (Admin, member, owner, etc)
Admin → kick a member out
member →
owner →

PBAC : Define dynamic rules / privileges → policies
policy → Some user can ~~access~~ access on paid Questions
new policy → No more three req per second

ACL : Highly granular control
Individual permissions :
turn on/off
For every user diff permissions can be on/off

Discord Server : 1 user : Message (tick) ✓

API End points :
register post www.insta.com/register

```
{ "username" : "aditya",
  "fullName" : " " ,
  "password" : " " ,
  "mobileNo" : " "
}
```

post /login

```
{ "username" : " ",
  "password" : " "
}
```

/logout
Mark user session as inactive (Revoke authentication)

other :
/create /{userId}
/grant /{userId} /{permissionId}
/revoke /{userId} /{permissionId}
/expire /{sessionId}
/validate /{sessionId}

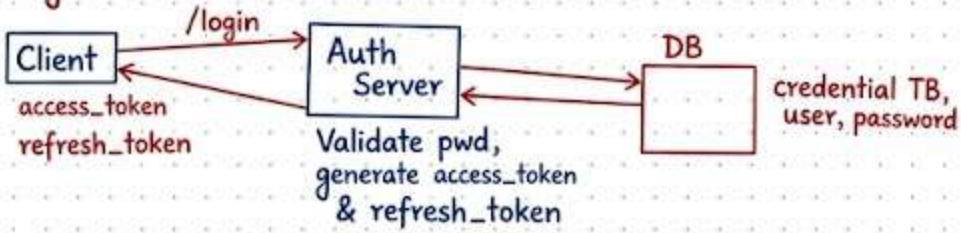
Table Structure :

- User Table : to store basic user info
  UserId , email , name , username , role , createdAt, updatedAt etc.
- Credential table ;
  Credential Id , email (FK) , access_token , refresh_token
- Password table :
  Password Id , email (FK) , hashed-password , salt, last_logon etc.
- Session table :
  Active sessions of user (history)
  SessionId , email (FK) , last_logged_in etc.

↳ User Registration :

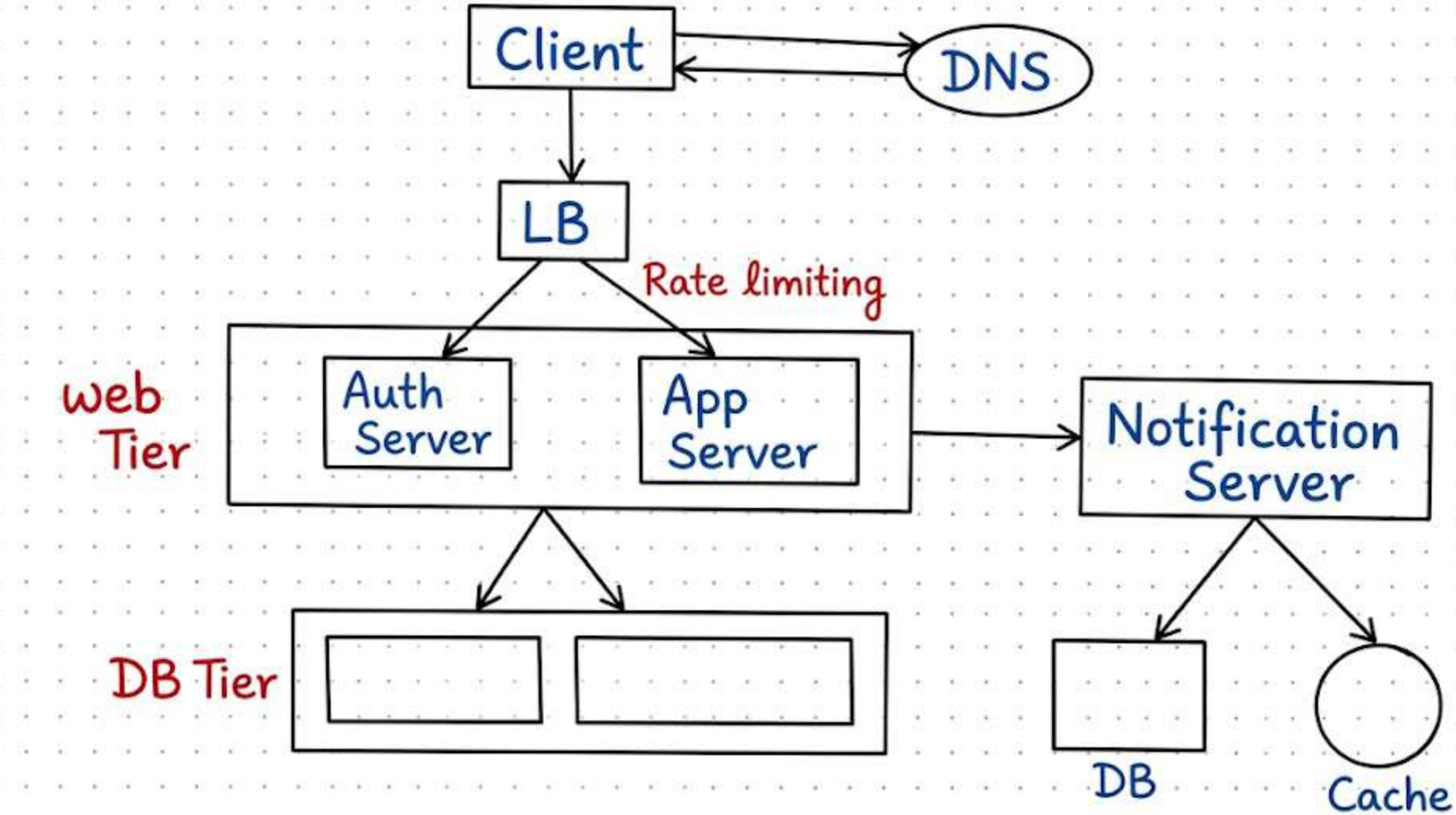


↳ Login User



↳ Session Management




Client
DNS
LB
Rate limiting
web Tier
Auth Server
App Server
Notification Server
DB Tier
DB
Cache

↳ Logout :


/logout
Client
Auth Server
DB
OK